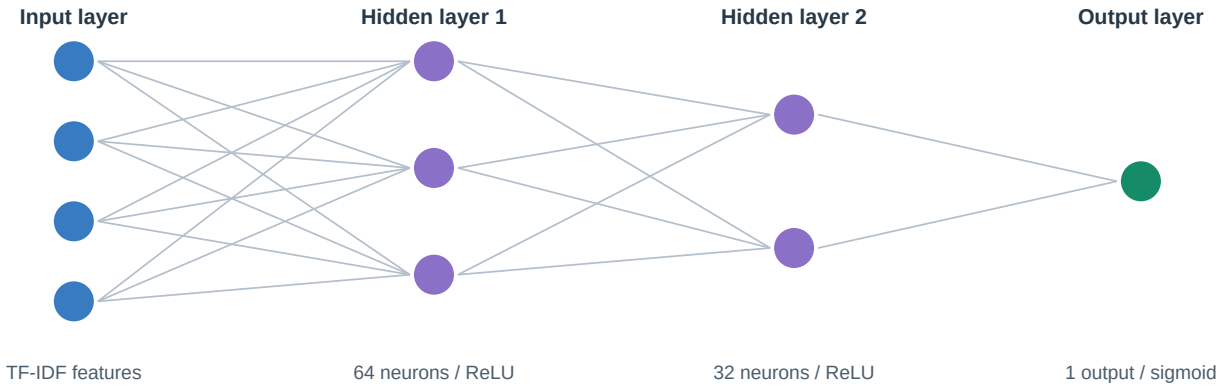
IMPLEMENTATION OF NEURAL NETWORK FOR TEXT CLASSIFICATION

Objective: Build an Artificial Neural Network (ANN) that classifies text reviews as Positive or Negative. The experiment uses TF-IDF to convert reviews into numerical features and a Multi-Layer Perceptron (MLP) to learn the classification task.

1. Execution Workflow

Stage	What happens
Input dataset	Read text reviews and their sentiment labels from sentiment_dataset.csv.
Text representation	Convert the reviews into numerical TF-IDF feature vectors.
Input layer	Receive the TF-IDF feature values.
Hidden layers	Learn patterns using 64 and 32 neurons with ReLU activation.
Output layer	Produce a probability used to classify the review as Positive or Negative.
Training	Backpropagation and the Adam optimizer update network weights to reduce prediction error.
Evaluation	Measure accuracy, precision, recall and F1-score; inspect the confusion matrix and loss curve.

2. ANN Architecture



The actual input layer has one input for each TF-IDF feature retained by the vectorizer. The diagram is conceptual and shows representative input neurons rather than every feature.

Layer	Role in this experiment
Input	Receives the TF-IDF numerical representation of a review.
Hidden layer 1	64 neurons; ReLU activation.
Hidden layer 2	32 neurons; ReLU activation.
Output	One binary-classification output; sigmoid-equivalent probability.

3. Key Concepts

ReLU: A hidden-layer activation function that introduces non-linearity. **Sigmoid:** Maps a binary-classification score to a value between 0 and 1. In scikit-learn's MLPClassifier, the binary output uses a logistic (sigmoid-equivalent) activation. **Backpropagation:** Computes how the prediction error relates to the network weights and passes this information backward so the weights can be adjusted during training.

4. Software Requirements

Use Python with Jupyter Notebook, Anaconda, VS Code or a terminal. Install the libraries:

```
pip install pandas numpy scikit-learn matplotlib seaborn
```

5. Dataset Preparation

Use the file **sentiment_dataset.csv** created for the previous experiment. It must contain two columns: **Review** (the text) and **Sentiment** (Positive or Negative). The practice dataset contains 100 synthetic reviews: 50 Positive and 50 Negative.

Example records: The product is excellent - Positive; Amazing quality and fast delivery - Positive; Very poor quality - Negative; The service was terrible - Negative.

Dataset note: This small synthetic dataset is suitable for learning the execution workflow. It is not large or diverse enough to support reliable claims about real-world ANN performance.

6. Complete Python Implementation

Save the following code as **ann_text_classification.py** in the same folder as **sentiment_dataset.csv**.

```
# IMPLEMENTATION OF NEURAL NETWORK FOR TEXT CLASSIFICATION

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix
)

# STEP 1: Load dataset
data = pd.read_csv("sentiment_dataset.csv")
X = data["Review"].astype(str)
y = data["Sentiment"].astype(str).str.strip().str.title()

# STEP 2: Split dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# STEP 3: Build TF-IDF + ANN pipeline
ann_pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(
        lowercase=True, stop_words="english", max_features=3000
    )),
    ("ann", MLPClassifier(
        hidden_layer_sizes=(64, 32),
        activation="relu",
        solver="adam",
        learning_rate_init=0.001,
        max_iter=500,
        random_state=42
    ))
])

# STEP 4: Train the neural network
print("Training ANN model...")
ann_pipeline.fit(X_train, y_train)
print("Training completed!")

# STEP 5: Predict the test data
y_pred = ann_pipeline.predict(X_test)

# STEP 6: Evaluate the model
print("\nANN MODEL PERFORMANCE")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(
    y_test, y_pred, average="weighted", zero_division=0
))
print("Recall:", recall_score(
    y_test, y_pred, average="weighted", zero_division=0
))
print("F1 Score:", f1_score(
    y_test, y_pred, average="weighted", zero_division=0
))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

# STEP 7: Display confusion matrix
cm = confusion_matrix(
    y_test, y_pred, labels=["Negative", "Positive"]
)
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)
plt.title("ANN Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

# STEP 8: Plot training loss
ann_model = ann_pipeline.named_steps["ann"]
plt.figure(figsize=(7, 4))
plt.plot(ann_model.loss_curve_, label="Training Loss")
plt.title("ANN Training Loss")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# STEP 9: Predict new reviews
new_reviews = [
    "This product is excellent and amazing",
    "The product is very poor and disappointing"
]
predictions = ann_pipeline.predict(new_reviews)
probabilities = ann_pipeline.predict_proba(new_reviews)

print("\nNEW REVIEW PREDICTIONS")
for review, prediction, probability in zip(
    new_reviews, predictions, probabilities
):
    print("\nReview:", review)
    print("Predicted Sentiment:", prediction)
    for label, score in zip(ann_model.classes_, probability):
        print(label, "Probability:", round(score, 4))
```

7. How to Execute the Program

Step 1: Create a folder named **ANN_Text_Classification**.

Step 2: Place these two files inside the folder:

File	Purpose
sentiment_dataset.csv	Contains the labelled review dataset.
ann_text_classification.py	Contains the ANN implementation.

Step 3: Open Command Prompt or the VS Code terminal and navigate to the folder:

```
cd path\to\ANN_Text_Classification
```

Step 4: Install the required libraries:

```
pip install pandas numpy scikit-learn matplotlib seaborn
```

Step 5: Run the program:

```
python ann_text_classification.py
```

Step 6: Observe the training message, evaluation metrics, confusion matrix, training loss curve and predictions for new reviews.

8. Important ANN Parameters

Parameter	Explanation
hidden_layer_sizes=(64, 32)	Creates two hidden layers: the first has 64 neurons and the second has 32 neurons.
activation='relu'	Uses ReLU activation in the hidden layers.
solver='adam'	Uses the Adam optimizer to update the network weights.
learning_rate_init=0.001	Sets the initial learning rate for weight updates.
max_iter=500	Sets the maximum number of training iterations.
random_state=42	Supports reproducible results for the same environment and data.
max_features=3000	Limits the TF-IDF vocabulary to at most 3,000 features.

9. Understanding the Output

Output	What it tells you
Accuracy	The proportion of test reviews classified correctly.
Precision	How often predictions for a class are correct.
Recall	How many actual examples of a class are identified.
F1-score	A combined measure of precision and recall.
Confusion matrix	Shows correct and incorrect predictions for Positive and Negative reviews.
Training loss curve	Shows how the model's loss changes over training iterations; a decreasing curve generally indicates learning.
Prediction probabilities	For each new review, gives the model's estimated probability for each class.

10. Suggested Laboratory Experiment

Title: Implementation of Neural Network for Text Classification

Aim: To implement an Artificial Neural Network using TF-IDF feature extraction for classifying textual reviews into positive and negative sentiments, and understand the role of input, hidden and output layers, activation functions and backpropagation.

Learning Outcomes

1. Understand the architecture of an Artificial Neural Network.
2. Implement input, hidden and output layers for text classification.
3. Explain the role of ReLU and sigmoid activation functions.
4. Understand how backpropagation updates network weights.
5. Evaluate the ANN using accuracy, precision, recall and F1-score.

Teaching sequence: Demonstrate TF-IDF conversion first, trace the values through the input and hidden layers, inspect the output probability, and then explain how backpropagation updates weights to reduce error. For meaningful evaluation, repeat the experiment with a larger and more diverse labelled dataset.